



S.E.P. TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

ACTIVIDAD:

Curso de agente ia

PRESENTAN:

LURIA SANTIAGO DANIEL 22350722

DOCENTE:

JULIO AGUILAR CARMONA

CARRERA:

INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

CIENCIAS DE DATOS

04/mayo/2026



Contenido

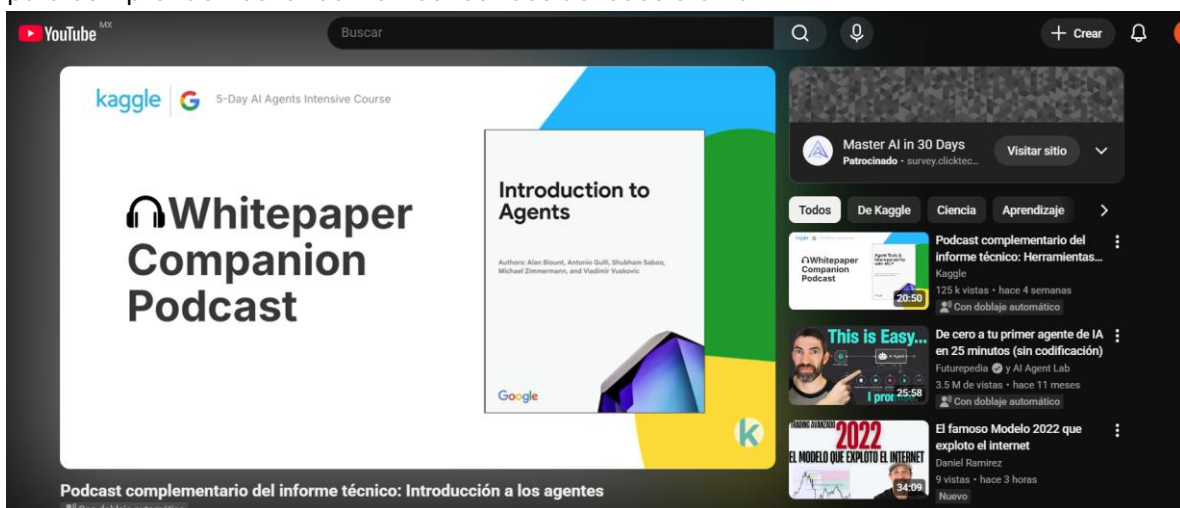
Introducción	3
Día 1 (Introducción a los agentes).....	4
MULTIAGENTE	10
Día 2 (Herramientas para agentes e interoperabilidad con el Protocolo de Contexto de Modelo (MCP))	15
Día 3 (Ingeniería de contexto: sesiones y memoria)	24
Día 3b - Memoria del agente	34
Día 4 depuración.....	42
DIA 5 Prototipos de producción	51

Introducción

El presente documento detalla el progreso técnico y práctico de un curso intensivo de cinco días centrado en el desarrollo y la implementación de agentes de Inteligencia Artificial, utilizando herramientas de Kaggle y Google. A lo largo del texto, se documenta una evolución estructurada que comienza con la configuración de entornos y la creación de agentes básicos, para luego escalar hacia sistemas más complejos. El contenido aborda áreas fundamentales como la orquestación de arquitecturas multiagente, la integración de herramientas externas y APIs para la interacción con el mundo real, la ingeniería de contexto con gestión de memoria persistente, así como la aplicación de técnicas de depuración, evaluación de errores y, finalmente, el despliegue de prototipos de producción.

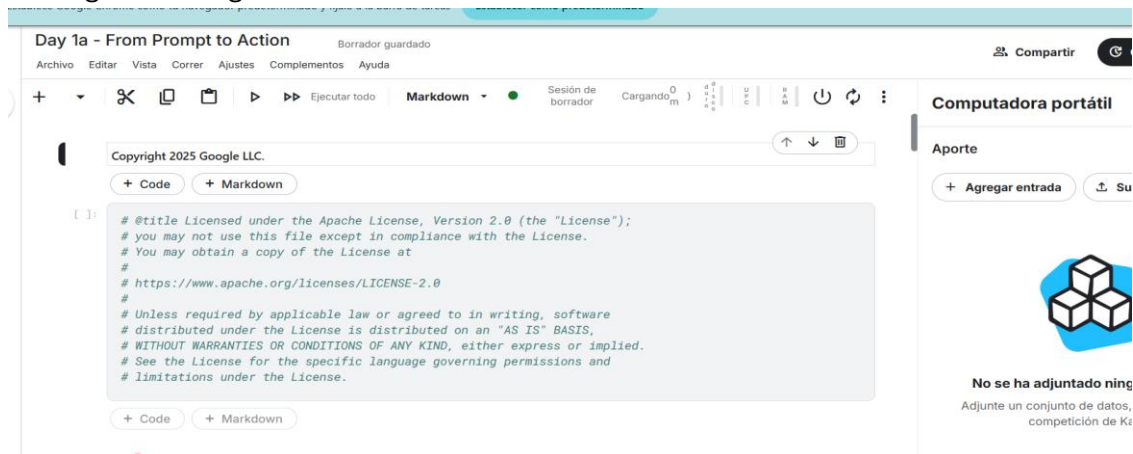
Día 1 (Introducción a los agentes)

Se inició el curso intensivo de 5 días sobre agentes de IA de Kaggle y Google, comenzando con el material teórico y el podcast complementario titulado "Introducción a los agentes" para comprender los fundamentos teóricos del ecosistema

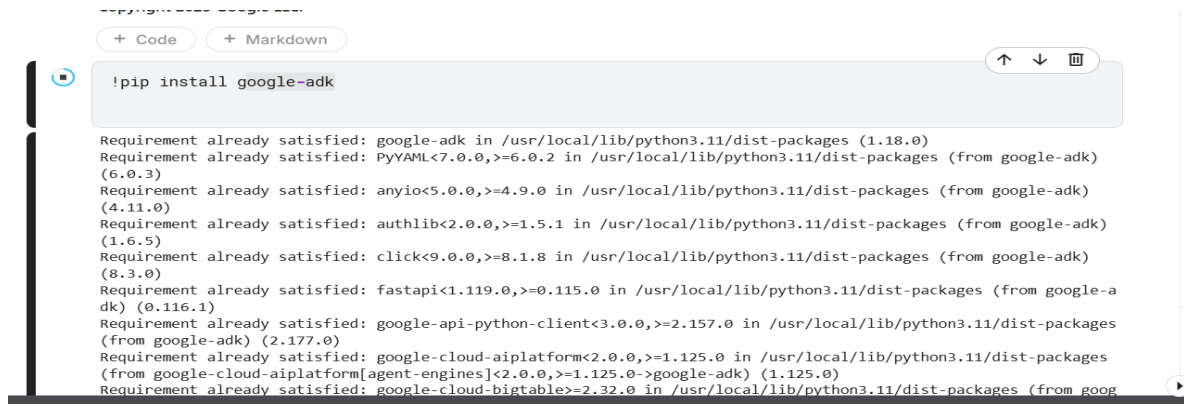


Se preparó el entorno de desarrollo en un cuaderno de Kaggle, estableciendo el título "Day 1a - From Prompt to Action" y revisando las licencias de software Apache 2.0 necesarias para el

uso legal del código.



Se procedió a la instalación del kit de desarrollo de agentes de Google mediante el comando `pip install google-adk`, verificando que todas las dependencias y librerías necesarias estuvieran correctamente satisfechas en el sistema.



Se realizó la configuración de seguridad integrando la API Key de Gemini a través de `UserSecretsClient` de Kaggle, confirmando mediante un mensaje de éxito que la

autenticación fue completada correctamente.

Run the cell below to complete authentication.

```
[9]: import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🔑 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets.
    ).

✅ Gemini API key setup complete.
```

Se importaron los componentes esenciales del ADK, incluyendo las clases para la creación de agentes, modelos de lenguaje de Google (Gemini), y herramientas de búsqueda para dotar de capacidades operativas al sistema.

1.3: Import ADK components

Now, import the specific components you'll need from the Agent Development Kit and the Generative AI library. This keeps your code organized and ensures we have access to the necessary building blocks.

+ Code

+ Markdown

```
from google.adk.agents import Agent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.tools import google_search
from google.genai import types

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

Se definieron funciones auxiliares y se configuró la interfaz de usuario web del ADK, visualizando mediante código HTML el botón de acceso para interactuar con los agentes de

forma gráfica.

```

<li style="margin-bottom: 5px;"><strong>Run the next cell</strong> (the one with <code>!
<li style="margin-bottom: 5px;">Wait for that cell to show it is "Running" (it will not
<li>Once it's running, <strong>return to this button</strong> and click it to open the U
</ol>
<em style="font-size: 0.9em; color: #555;">(If you click the button before running the next
</div>
<a href='{url}' target='_blank' style="
    display: inline-block; background-color: #1a73e8; color: white; padding: 10px 20px;
    text-decoration: none; border-radius: 25px; font-family: sans-serif; font-weight: 500;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2); transition: all 0.2s ease;">
    Open ADK Web UI (after running cell below) ↗
</a>
</div>
"""

display(HTML(styled_html))

return url_prefix

print("✅ Helper functions defined.")

```

✅ Helper functions defined.

Se definió el primer agente base, denominado "helpful_assistant", utilizando el modelo gemini-2.5-flash-lite y configurando instrucciones específicas para que actúe como un asistente útil con acceso a Google Search.

```

root_agent = Agent(
    name="helpful_assistant",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    description="A simple agent that can answer general questions.",
    instruction="You are a helpful assistant. Use Google Search for current info or if unsure.",
    tools=[google_search],
)

print("✅ Root Agent defined.")

```

✅ Root Agent defined.

+ Code + Markdown

2.3 Run your agent

Se creó una instancia de InMemoryRunner vinculada al agente raíz, lo que permite ejecutar y probar las capacidades del asistente dentro de la memoria de la sesión actual.

```

[21]: runner = InMemoryRunner(agent=root_agent)

print("✅ Runner created.")

```

✅ Runner created.

Se realizó la primera prueba de depuración consultando qué es el ADK de Google; el agente respondió con precisión detallando que es un framework de código abierto disponible en

Python, Go, Java y TypeScript.

```
> response = await runner.run_debug(
    "What is Agent Development Kit from Google? What languages is the SDK available in?"
)

### Created new session: debug_session_id

User > What is Agent Development Kit from Google? What languages is the SDK available in?
helpful_assistant > The Agent Development Kit (ADK) from Google is an open-source framework designed to simplify the development, debugging, and deployment of AI agents at an enterprise scale. It enables developers to build everything from simple AI assistants to complex, mission-critical business workflows and multi-agent systems. The ADK provides a comprehensive suite of tools for building, running, evaluating, and scaling AI architectures, allowing for flexible orchestration and the integration of third-party applications and custom code.

The SDK is available in the following programming languages:
* Python
* Go
* Java
* TypeScript
```

+ Code + Markdown

En una prueba de interacción más informal, se evaluó la capacidad del agente para ofrecer consejos estratégicos y motivacionales ante consultas sobre cómo aprobar materias escolares.

```
> response = await runner.run_debug(
    "como aprobar la materia si no hice nada?"
)

### Continue session: debug_session_id

User > como aprobar la materia si no hice nada?
helpful_assistant > **Estrategias para aprobar una materia sin haber hecho nada:**

Si te encuentras en una situación en la que no has dedicado tiempo a una materia y necesitas aprobarla, existen algunas estrategias que podrías intentar, aunque es importante recordar que la mejor manera de tener éxito es el estudio constante.

**Antes del examen:**

* **Prioriza lo esencial:** Identifica los temas o conceptos más importantes que probablemente aparecerán en el examen. Si tienes acceso a exámenes anteriores, revisa cuáles son las preguntas que se repiten con frecuencia.
* **Resúmenes y esquemas:** Crea resúmenes concisos o esquemas de los temas clave. Esto te dará una visión general y te ayudará a organizar la información de manera más manejable.
* **Guías de estudio exprés:** Si existen guías de estudio o resúmenes creados por el profesor o compañeros, utilízalas. Estas suelen enfocarse en los puntos más relevantes.
* **Aprovecha las clases:** Si aún tienes clases antes del examen, presta máxima atención. Siéntate adelante para minimizar distracciones y trata de entender los puntos que el profesor enfatiza. Pregunta dudas que surjan.
```

Se utilizó la interfaz de línea de comandos (CLI) con el comando `!adk create` para generar la estructura de archivos de un nuevo agente de muestra, creando automáticamente los

archivos .env, __init__.py y agent.py

```
+ Code + Markdown
```

```
!adk create sample-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

```
Agent created in /kaggle/working/sample-agent:  
- .env  
- __init__.py  
- agent.py
```

```
+ Code + Markdown
```

Get your custom URL to access the ADK web UI in the Kaggle Notebooks environment:

Se inició el servidor web del ADK para pruebas locales, estableciendo un proceso de servidor FastAPI en el puerto 8000 para habilitar la interfaz de usuario del kit de desarrollo.

```
+ Code + Markdown
```

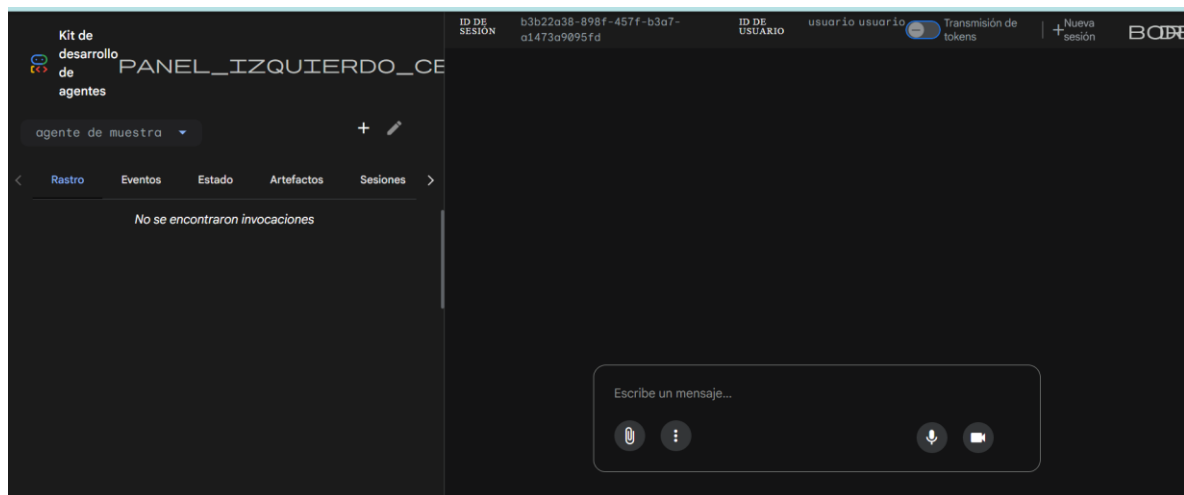
```
!adk web --url_prefix {url_prefix}
```

```
/usr/local/lib/python3.11/dist-packages/google/adk/cli/fast_api.py:130: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.  
credential_service = InMemoryCredentialService()  
/usr/local/lib/python3.11/dist-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.  
super().__init__()  
INFO: Started server process [337]  
INFO: Waiting for application startup.
```

```
+-----+  
| ADK Web Server started  
| For local testing, access at http://127.0.0.1:8000.  
+-----+
```

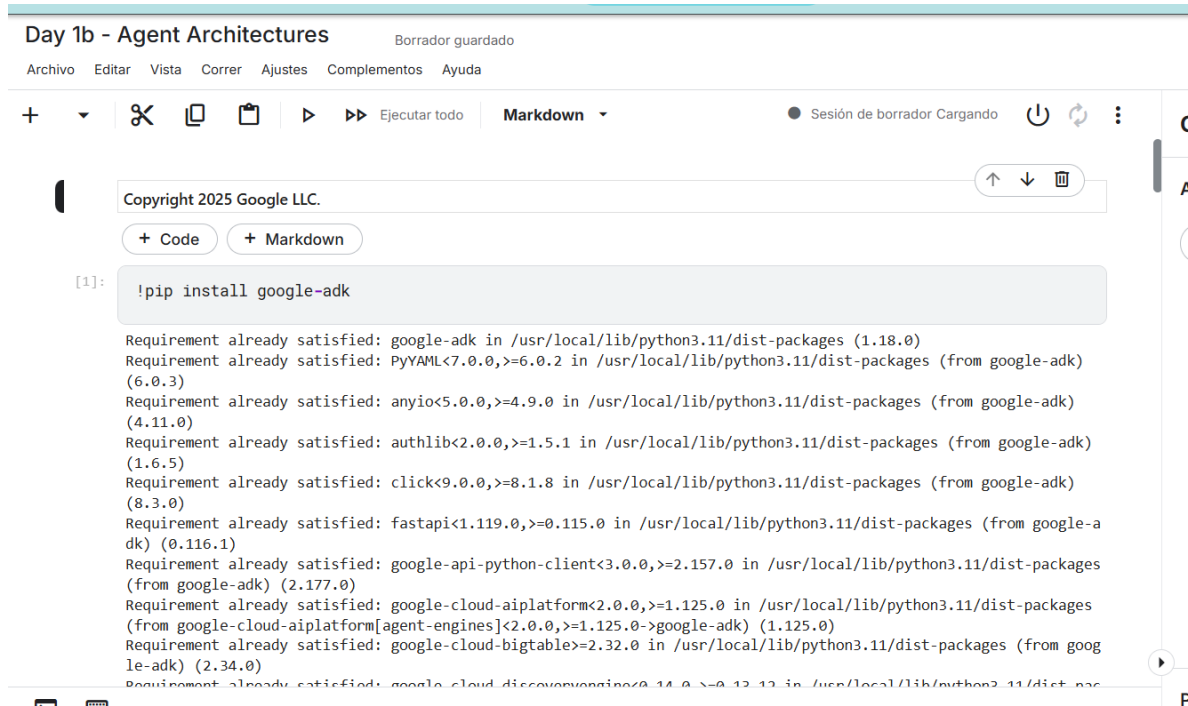
```
INFO: Application startup complete.
```

Se accedió al panel de control del Kit de Desarrollo de Agentes, donde se observa la interfaz oscura con secciones para el rastreo de invocaciones, eventos, estado y gestión de sesiones de chat.

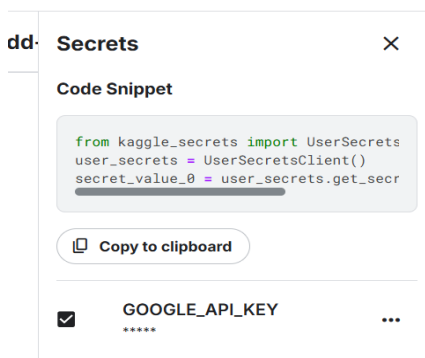


MULTIAGENTE

Se inició la segunda parte del primer día enfocada en "Agent Architectures", repitiendo la validación de las librerías google-adk para asegurar un entorno de ejecución consistente para sistemas de múltiples agentes.



Se gestionó nuevamente el panel de secretos de Kaggle para asegurar que la variable `GOOGLE_API_KEY` estuviera seleccionada y disponible para el nuevo cuaderno de arquitecturas.



Se ejecutó el bloque de código para la vinculación de la clave de API de Gemini, validando que el entorno estuviera listo para realizar llamadas a los modelos de lenguaje a gran escala.

Run the cell below to complete authentication.

+ Code

+ Markdown

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🔥 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets"
    )
```

✅ Gemini API key setup complete.

+ Code

+ Markdown

1.3: Import ADK components

Para esta arquitectura avanzada, se importaron clases adicionales como SequentialAgent, ParallelAgent y LoopAgent, ampliando las posibilidades de flujo de trabajo entre diferentes entidades de IA.

```
from google.adk.agents import Agent, SequentialAgent, ParallelAgent, LoopAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import InMemoryRunner
from google.adk.tools import AgentTool, FunctionTool, google_search
from google.genai import types

print("✅ ADK components imported successfully.")
```

✅ ADK components imported successfully.

Se configuraron las opciones de reintento (HttpRetryOptions) con un máximo de 5 intentos y retroceso exponencial para manejar errores transitorios de red o límites de tasa de la API.

1.4: Configure Retry Options

When working with LLMs, you may encounter transient errors like rate limits or temporary service unavailability. Retry options automatically handle these failures by retrying the request with exponential backoff.

+ Code

+ Markdown

```
retry_config=types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)
```

+ Code

+ Markdown

Se creó el "Research Agent", un agente especializado cuya única tarea es utilizar la herramienta de búsqueda de Google para encontrar piezas de información relevante sobre temas específicos.

```
[18]: # Research Agent: Its job is to use the google_search tool and present findings.
research_agent = Agent(
    name="ResearchAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""You are a specialized research agent. Your only job is to use the
google_search tool to find 2-3 pieces of relevant information on the given topic and present the fin
tools=[google_search],
    output_key="research_findings", # The result of this agent will be stored in the session state with
)

print("✅ research_agent created.")
```

✅ research_agent created.

Se definió el "Summarizer Agent", encargado de recibir los hallazgos de investigación y transformarlos en un resumen conciso en formato de lista con puntos clave.

```
# Summarizer Agent: Its job is to summarize the text it receives.
summarizer_agent = Agent(
    name="SummarizerAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # The instruction is modified to request a bulleted list for a clear output format.
    instruction="""Read the provided research findings: {research_findings}
Create a concise summary as a bulleted list with 3-5 key points.""",
    output_key="final_summary",
)

print("✅ summarizer_agent created.")
```

✅ summarizer_agent created.

Se estableció el "Research Coordinator", un agente raíz que orquesta el flujo de trabajo llamando al agente de investigación y luego al sintetizador para entregar una respuesta final estructurada.

```
[26]: # Root Coordinator: Orchestrates the workflow by calling the sub-agents as tools.
root_agent = Agent(
    name="ResearchCoordinator",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # This instruction tells the root agent HOW to use its tools (which are the other agents).
    instruction="""You are a research coordinator. Your goal is to answer the user's query by orchestrat
1. First, you MUST call the 'ResearchAgent' tool to find relevant information on the topic provided by t
2. Next, after receiving the research findings, you MUST call the 'SummarizerAgent' tool to create a con
3. Finally, present the final summary clearly to the user as your response.""",
    # We wrap the sub-agents in 'AgentTool' to make them callable tools for the root agent.
    tools=[AgentTool(research_agent), AgentTool(summarizer_agent)],
)

print("✅ root_agent created.")
```

✅ root_agent created.

Se ejecutó una prueba compleja sobre avances en computación cuántica; el sistema coordinó exitosamente la búsqueda y síntesis, proporcionando una respuesta detallada sobre la potencia computacional y algoritmos.

```
[27]: runner = InMemoryRunner(agent=root_agent)
      response = await runner.run_debug(
          "What are the latest advancements in quantum computing and what do they mean for AI?"
      )

### Created new session: debug_session_id

User > What are the latest advancements in quantum computing and what do they mean for AI?
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
ResearchCoordinator > Recent advancements in quantum computing, particularly in computational power and novel algorithms, are poised to revolutionize AI. Quantum computers, with their ability to perform massive parallel processing using qubits, can significantly accelerate AI model training and tackle complex problems currently beyond the reach of classical computers. This enhanced power is expected to lead to more efficient and accurate AI models, improved data analysis, and breakthroughs in fields like drug discovery and climate modeling. The development of hybrid quantum-classical approaches and more accessible quantum hardware and software further pave the way for augmented AI capabilities in the near future.
```

Se configuró un "Outline Agent" especializado en la creación de esquemas para publicaciones de blog, definiendo requisitos como titulares llamativos, ganchos de introducción y secciones con puntos clave.

```
▶ # Outline Agent: Creates the initial blog post outline.
outline_agent = Agent(
    name="OutlineAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    instruction="""Create a blog outline for the given topic with:
1. A catchy headline
2. An introduction hook
3. 3-5 main sections with 2-3 bullet points for each
4. A concluding thought""",
    output_key="blog_outline", # The result of this agent will be stored in the session state with this key
)

print("✅ outline_agent created.")

✅ outline_agent created.
```

Finalmente, se creó el "Writer Agent", diseñado para tomar el esquema generado previamente y redactar un borrador completo de blog de entre 200 y 300 palabras con un tono informativo.

⬢

↑ ↓ 🗑

```
# Writer Agent: Writes the full blog post based on the outline from the previous agent.
writer_agent = Agent(
    name="WriterAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # The `{blog_outline}` placeholder automatically injects the state value from the previous agent's output.
    instruction="""Following this outline strictly: {blog_outline}
    Write a brief, 200 to 300-word blog post with an engaging and informative tone.""",
    output_key="blog_draft", # The result of this agent will be stored with this key.
)

print("✅ writer_agent created.")
```

✅ writer_agent created.

⋮ ⋮ ⋮ ⋮ ⋮

Aquí se muestra la ejecución de una parte del código del agente. Se están utilizando herramientas y modelos para realizar una tarea específica. Esto permite validar que el agente responde correctamente.

⬢

↑ ↓ 🗑

```
# Editor Agent: Edits and polishes the draft from the writer agent.
editor_agent = Agent(
    name="EditorAgent",
    model=Gemini(
        model="gemini-2.5-flash-lite",
        retry_options=retry_config
    ),
    # This agent receives the `{blog_draft}` from the writer agent's output.
    instruction="""Edit this draft: {blog_draft}
    Your task is to polish the text by fixing any grammatical errors, improving the flow and sentence structure.
    output_key="final_blog", # This is the final output of the entire pipeline.
)

print("✅ editor_agent created.")
```

✅ editor_agent created.

+ Code + Markdown

Finalmente, se creó el "Writer Agent", diseñado para tomar el esquema generado previamente y redactar un borrador completo de blog de entre 200 y 300 palabras con un tono informativo.

```
runner = InMemoryRunner(agent=root_agent)
response = await runner.run_debug(
    "Write a blog post about the benefits of multi-agent systems for software developers"
)
```

```
### Created new session: debug_session_id
```

```
User > Write a blog post about the benefits of multi-agent systems for software developers
OutlineAgent > Okay, I am OutlineAgent. Here is a blog outline for the topic "Benefits of Multi-Agent Systems for Software Developers":
```

```
## Blog Outline:
```

```
**Headline:** Supercharge Your Code: Why Multi-Agent Systems Are Your Next Developer Power-Up
```

```
**Introduction Hook:**
```

```
* Ever felt like your software projects are sprawling, complex beasts that are getting harder and harder to tame? What if there was a way to break down these behemoths into smaller, smarter, and more collaborative units? Enter the world of Multi-Agent Systems (MAS).
```

```
**Main Sections:**
```

Día 2 (Herramientas para agentes e interoperabilidad con el Protocolo de Contexto de Modelo (MCP))

Se inició la sesión del segundo día titulada "Day 2 - Agentic Tools", centrada en cómo permitir que los agentes interactúen con el mundo real mediante la ejecución de código y llamadas a servicios externos.

Dentro de este código creamos una función para simular estructuras de comisión de conversión de para diferentes métodos de pagos

```
Returns:
    Dictionary with status and fee information.
    Success: {"status": "success", "fee_percentage": 0.02}
    Error: {"status": "error", "error_message": "Payment method not found"}
"""
# This simulates looking up a company's internal fee structure.
fee_database = {
    "platinum credit card": 0.02, # 2%
    "gold debit card": 0.035, # 3.5%
    "bank transfer": 0.01, # 1%
}

fee = fee_database.get(method.lower())
if fee is not None:
    return {"status": "success", "fee_percentage": fee}
else:
    return {
        "status": "error",
        "error_message": f"Payment method '{method}' not found",
    }

print("✅ Fee lookup function created")
print(f"🔍 Test: {get_fee_for_payment_method('platinum credit card')}")

✅ Fee lookup function created
🔍 Test: {"status": "success", "fee_percentage": 0.02}
```

En esta función implementamos un simulador de tasas de cambio entre divisas siguiendo el estándar ISO-4217

```
rate_database = {
    "usd": {
        "eur": 0.93, # Euro
        "jpy": 157.58, # Japanese Yen
        "inr": 83.58, # Indian Rupee
    }
}

# Input validation and processing
base = base_currency.lower()
target = target_currency.lower()

# Return structured result with status
rate = rate_database.get(base, {}).get(target)
if rate is not None:
    return {"status": "success", "rate": rate}
else:
    return {
        "status": "error",
        "error_message": f"Unsupported currency pair: {base_currency}/{target_currency}",
    }

print("✅ Exchange rate function created")
print(f"🔍 Test: {get_exchange_rate('USD', 'EUR')}")
```

```
✅ Exchange rate function created
🔍 Test: {'status': 'success', 'rate': 0.93}
```

Con este código creamos el agente que se conectara a la herramienta de cambio de divisa

```
# Currency agent with custom function tools
currency_agent = LLMAgent(
    name="currency_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="""You are a smart currency conversion assistant.

For currency conversion requests:
1. Use 'get_fee_for_payment_method()' to find transaction fees
2. Use 'get_exchange_rate()' to get currency conversion rates
3. Check the 'status' field in each tool's response for errors
4. Calculate the final amount after fees based on the output from 'get_fee_for_payment_method' and 'get_exchange_rate' methods and provide a clear breakdown
5. First, state the final converted amount.
   Then, explain how you got that result by showing the intermediate amounts. Your explanation must include: the fee percentage and its value in the original currency, the amount remaining after the fee, and the exchange rate used for the final conversion.

If any tool returns status 'error', explain the issue to the user clearly.
""",
    tools=[get_fee_for_payment_method, get_exchange_rate],
)

print("✅ Currency agent created with custom function tools")
print("🔍 Available tools:")
print("• get_fee_for_payment_method - Looks up company fee structure")
print("• get_exchange_rate - Gets current exchange rates")
```

```
✅ Currency agent created with custom function tools
🔍 Available tools:
• get_fee_for_payment_method - Looks up company fee structure
```

Aquí podemos ver como si le el cambio de 500 dólares a euros con una tarjeta de crédito nos dice de cuanto seria el cambio de la divisa

```
# Test the currency agent
currency_runner = InMemoryRunner(agent=currency_agent)
_ = await currency_runner.run_debug(
    "I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?"
)
```

```
### Created new session: debug_session_id
```

```
User > I want to convert 500 US Dollars to Euros using my Platinum Credit Card. How much will I receive?
```

```
WARNING:google.genai.types:Warning: there are non-text parts in the response: ['function_call', 'function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
WARNING:google.genai.types:Warning: there are non-text parts in the response: ['function_call', 'function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
```

```
currency_agent > You will receive €465.40.
```

```
Here's how that's calculated:
```

1. The transaction fee for using a platinum credit card is 2%, which is \$10.00.
 2. This leaves you with \$490.00 to convert.
 3. The exchange rate from USD to EUR is 0.93, so \$490.00 becomes €455.70.
- Please note that the fee was deducted in the original currency (USD).

Con este código creamos un agente de calculadora para que tenga mayor precisión al momento de la conversión

```
calculation_agent = LlmAgent(
    name="CalculationAgent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="""You are a specialized calculator that ONLY responds with Python code. You are forbidden from providing any text, explanations, or conversions.

    Your task is to take a request for a calculation and translate it into a single block of Python code that calculates the answer.

    **RULES:**
    1. Your output MUST be ONLY a Python code block.
    2. Do NOT write any text before or after the code block.
    3. The Python code MUST calculate the result.
    4. The Python code MUST print the final result to stdout.
    5. You are PROHIBITED from performing the calculation yourself. Your only job is to generate the code that will perform the calculation.

    Failure to follow these rules will result in an error.
    """,
    code_executor=BuiltInCodeExecutor(), # Use the built-in Code Executor Tool. This gives the agent code execution capabilities
)
```

En este código actualizado utilizamos 2 instrucciones el primero que utiliza el primer modelo de herramienta que hace la conversión, pero no es precisa y el mejorado que utiliza la herramienta del calculation_agent

```
5. Provide Detailed Breakdown: In your summary, you must:
    * State the final converted amount.
    * Explain how the result was calculated, including:
      * The fee percentage and the fee amount in the original currency.
      * The amount remaining after deducting the fee.
      * The exchange rate applied.
    """
    tools=[
        get_fee_for_payment_method,
        get_exchange_rate,
        AgentTool(agent=calculation_agent), # Using another agent as a tool!
    ],
)

print("✅ Enhanced currency agent created")
print("🔗 New capability: Delegates calculations to specialist agent")
print("🔧 Tool types used:")
print("    • Function Tools (fees, rates)")
print("    • Agent Tool (calculation specialist)")
```

```
✅ Enhanced currency agent created
🔗 New capability: Delegates calculations to specialist agent
🔧 Tool types used:
    • Function Tools (fees, rates)
    • Agent Tool (calculation specialist)
```

En este código ejecutamos el agente para el convertidor de divisa

```
# Test the enhanced agent
response = await enhanced_runner.run_debug(
    "Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation."
)

### Continue session: debug_session_id

User > Convert 1,250 USD to INR using a Bank Transfer. Show me the precise calculation.
```

En este código conectamos a una herramienta para extraer imágenes podemos tener otras, pero trabajaremos con esta

```
# MCP integration with Everything Server
mcp_image_server = McpToolset(
    connection_params=StdioConnectionParams(
        server_params=StdioServerParameters(
            command="npx", # Run MCP server via npx
            args=[
                "-y", # Argument for npx to auto-confirm install
                "@modelcontextprotocol/server-everything",
            ],
            tool_filter=["getTinyImage"],
        ),
        timeout=30,
    )
)

print("✅ MCP Tool created")
```

✅ MCP Tool created

+ Code + Markdown

En este código creamos un agente para la gestión de creación de imágenes pequeñas

```
# Create image agent with MCP integration
image_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="image_agent",
    instruction="Use the MCP Tool to generate images for user queries",
    tools=[mcp_image_server],
)
```

+ Code + Markdown

Aquí creamos el iniciador del generador de imágenes

```
from google.adk.runners import InMemoryRunner

runner = InMemoryRunner(agent=image_agent)
```

+ Code + Markdown

Aquí hacemos que el agente cree una imagen sencilla

```
response = await runner.run_debug("Provide a sample tiny image", verbose=True)
```

```
### Created new session: debug_session_id

User > Provide a sample tiny image

/usr/local/lib/python3.11/dist-packages/google/adk/tools/mcp_tool/mcp_tool.py:101: UserWarning: [EXPERIMENTAL] BaseAuthenticatedTool: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__(
WARNING:google_genai.types:Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
image_agent > [Calling tool: get-tiny-image({})]
image_agent > [Tool result: {'content': [{'type': 'text', 'text': "Here's the image you requested:"}, {'type': 'image', 'data': ...}]
image_agent > Here's the image you requested:
The image above is the MCP logo.
```

En este código hacemos que la imagen se decodifique y se muestre usando lpython display

A screenshot of a Jupyter Notebook code cell. The code is written in Python and is designed to process a response object, likely from a REST API, to find and display an image. The code uses the IPython.display module for displaying the image. The code is as follows:

```
from IPython.display import display, Image as IPIImage
import base64

for event in response:
    if event.content and event.content.parts:
        for part in event.content.parts:
            if hasattr(part, "function_response") and part.function_response:
                for item in part.function_response.response.get("content", []):
                    if item.get("type") == "image":
                        display(IPIImage(data=base64.b64decode(item["data"])))
```

The code cell has a run button (a right-pointing triangle) on the left and a toolbar with up, down, and delete icons on the right. The code is color-coded: keywords are green, strings are blue, and function names are red.

En esta creamos dos funciones uno que indique la aprobación humana para continuar y otro que lea la aprobación

```

LARGE_ORDER_THRESHOLD = 5

def place_shipping_order(
    num_containers: int, destination: str, tool_context: ToolContext
) -> dict:
    """Places a shipping order. Requires approval if ordering more than 5 containers (LARGE_ORDER_THRESHOLD).

    Args:
        num_containers: Number of containers to ship
        destination: Shipping destination

    Returns:
        Dictionary with order status
    """

    # -----
    #
    # SCENARIO 1: Small orders (<=5 containers) auto-approve
    if num_containers <= LARGE_ORDER_THRESHOLD:
        return {
            "status": "approved",
            "order_id": f"ORD-{num_containers}-AUTO",
            "num_containers": num_containers,
            "destination": destination,
            "message": f"Order auto-approved: {num_containers} containers to {destination}",
        }

    # -----
    #
    # SCENARIO 2: This is the first time this tool is called. Large orders need human approval - PAUSE here.
    if not tool_context.tool_confirmation:
        tool_context.request_confirmation(
            hint=f"⚠️ Large order: {num_containers} containers to {destination}. Do you want to approve?",
            payload={"num_containers": num_containers, "destination": destination},
        )
        return { # This is sent to the Agent
            "status": "pending",
            "message": f"Order for {num_containers} containers requires approval",
        }

    # -----
    #
    # SCENARIO 3: The tool is called AGAIN and is now resuming. Handle approval response - RESUME here.
    if tool_context.tool_confirmation.confirmed:
        return {
            "status": "approved",
            "order_id": f"ORD-{num_containers}-HUMAN",
            "num_containers": num_containers,
            "destination": destination,
            "message": f"Order approved: {num_containers} containers to {destination}",
        }
    else:
        return {
            "status": "rejected",
            "message": f"Order rejected: {num_containers} containers to {destination}",
        }

print("✅ Long-running functions created!")
```

✅ Long-running functions created!

En este código creamos el agente que cuando el pedido se mayor a 5 contenedores pedirá la aprobación humana si es menor la aprobación será de manera automática

```
# Create shipping agent with pausable tool
shipping_agent = LlmAgent(
    name="shipping_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="""You are a shipping coordinator assistant.

    When users request to ship containers:
    1. Use the place_shipping_order tool with the number of containers and destination
    2. If the order status is 'pending', inform the user that approval is required
    3. After receiving the final result, provide a clear summary including:
       - Order status (approved/rejected)
       - Order ID (if available)
       - Number of containers and destination
    4. Keep responses concise but informative
    """,
    tools=[FunctionTool(func=place_shipping_order)],
)

print("✅ Shipping Agent created!")
```

✅ Shipping Agent created!

En este código creamos un agente que guarde los datos del pedido cuando el agente es pausado para esperar la aprobación ya que sin eso no guardaría el pedido

```
# Wrap the agent in a resumable app - THIS IS THE KEY FOR LONG-RUNNING OPERATIONS!
shipping_app = App(
    name="shipping_coordinator",
    root_agent=shipping_agent,
    resumability_config=ResumabilityConfig(is_resumable=True),
)

print("✅ Resumable app created!")
```

✅ Resumable app created!

Aquí cuando ejecutamos el código conectamos el ejecutador a la herramienta que nos permite reanudar la operación para el agente sepa que se puede reanudar

```
session_service = InMemorySessionService()

# Create runner with the resumable app
shipping_runner = Runner(
    app=shipping_app, # Pass the app instead of the agent
    session_service=session_service,
)

print("✅ Runner created!")
```

✅ Runner created!

+ Code + Markdown

En este código vemos como los eventos funciona, como se pausa el agente para esperar la confirmación humana y como busca el id del evento que debe reanudarse

```
def check_for_approval(events):
    """Check if events contain an approval request.

    Returns:
        dict with approval details or None
    """
    for event in events:
        if event.content and event.content.parts:
            for part in event.content.parts:
                if (
                    part.function_call
                    and part.function_call.name == "adk_request_confirmation"
                ):
                    return {
                        "approval_id": part.function_call.id,
                        "invocation_id": event.invocation_id,
                    }
    return None
```

Este código nos ayuda a imprimir y extraer texto de los eventos

```
def print_agent_response(events):
    """Print agent's text responses from events."""
    for event in events:
        if event.content and event.content.parts:
            for part in event.content.parts:
                if part.text:
                    print(f"Agent > {part.text}")
```

En este código creamos una función que toma la decisión de verdad o falso y lo devuelve al agente

```
def create_approval_response(approval_info, approved):
    """Create approval response message."""
    confirmation_response = types.FunctionResponse(
        id=approval_info["approval_id"],
        name="adk_request_confirmation",
        response={"confirmed": approved},
    )
    return types.Content(
        role="user", parts=[types.Part(function_response=confirmation_response)]
    )

print("✅ Helper functions defined")
```

Dentro de este código juntamos cada uno de los eventos y las funciones y así poder coordinarlos

```
async def run_shipping_workflow(query: str, auto_approve: bool = True):
    """Runs a shipping workflow with approval handling.

    Args:
        query: User's shipping request
        auto_approve: Whether to auto-approve large orders (simulates human decision)
    """

    print(f"\n{'='*60}")
    print(f"User > {query}\n")

    # Generate unique session ID
    session_id = f"order_{uuid.uuid4().hex[:8]}"

    # Create session
    await session_service.create_session(
        app_name="shipping_coordinator", user_id="test_user", session_id=session_id
    )

    query_content = types.Content(role="user", parts=[types.Part(text=query)])
    events = []

    # -----
    # STEP 1: Send initial request to the Agent. If run_containers > 5, the Agent returns the special 'ack_request_confirmation' event
    async for event in shipping_runner.run_async(
        user_id="test_user", session_id=session_id, new_message=query_content
    ):
        events.append(event)

    # -----
    # STEP 2: Loop through all the events generated and check if 'ack_request_confirmation' is present.
    approval_info = check_for_approval(events)

    # -----
    # STEP 3: If the event is present, it's a large order - HANDLE APPROVAL WORKFLOW
    if approval_info:
        print(f"⏸ Pausing for approval...")
        print(f"Human Decision: ('APPROVE ✅' if auto_approve else 'REJECT ❌')\n")

        # PATH A: Resume the agent by calling run_async() again with the approval decision
        async for event in shipping_runner.run_async(
            user_id="test_user",
            session_id=session_id,
            new_message=create_approval_response(
                approval_info, auto_approve
            ), # Send human decision here
            invocation_id=approval_info["invocation_id"], # Critical: same invocation_id tells ADK to RESUME
        ):
            if event.content and event.content.parts:
                for part in event.content.parts:
                    if part.text:
                        print(f"Agent > {part.text}")

    # -----
    else:
        # PATH B: If the 'ack_request_confirmation' is not present - no approval needed - order completed immediately.
        print_agent_response(events)

    print(f"\n{'='*60}")

    print("✅ Workflow function ready")
```

✅ Workflow function ready

Y que funcione en conjunto

Aquí vemos como funciona todo el flujo de trabajo cuando el pedido es menor a 5 la aprobación es automática y si es mayor pausa todo hasta que el usuario de la aprobación

después continua con la ejecución y que pasa cuando es rechazado

```
# Demo 1: It's a small order. Agent receives auto-approved status from tool
await run_shipping_workflow('Ship 3 containers to Singapore')

# Demo 2: Workflow simulates human decision: APPROVE
await run_shipping_workflow('Ship 10 containers to Rotterdam', auto_approve=True)

# Demo 3: Workflow simulates human decision: REJECT
await run_shipping_workflow('Ship 8 containers to Los Angeles', auto_approve=False)
```

User > Ship 3 containers to Singapore

WARNING:google_genai.types.warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

Agent > I have approved your order for 3 containers to Singapore. The order ID is ORD-3-AUTO.

User > Ship 10 containers to Rotterdam

WARNING:google_genai.types.warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

/usr/local/lib/python3.11/dist-packages/google/adk/tools/tool_context.py:92: UserWarning: [EXPERIMENTAL] ToolConfirmation: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

ToolConfirmation(
 /usr/local/lib/python3.11/dist-packages/google/adk/agents/invocation_context.py:298: UserWarning: [EXPERIMENTAL] BaseAgentState: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

self.agent_states[event.author] = BaseAgentState())

Pausing for approval...
Human Decision: APPROVE

Agent > Shipping order approved. Order ID: ORD-10-HUMAN. 10 containers will be shipped to Rotterdam.

User > Ship 8 containers to Los Angeles

WARNING:google_genai.types.warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

Pausing for approval...
Human Decision: REJECT

Agent > I am sorry, but your order for 8 containers to Los Angeles has been rejected. We can only ship a maximum of 5 containers per order. Please let me know if you would like to place a smaller order.

Día 3 (Ingeniería de contexto: sesiones y memoria)

Aquí se muestra la ejecución de una parte del código del agente. Se están utilizando herramientas y modelos para realizar una tarea específica. Esto permite validar que el agente responde correctamente.

Day 3a - Agent SessionsBorrador guardado

Archivo Editar Vista Correr Ajustes Complementos Ayuda

+ 🔍 📄 📌 ▶ ⏮ Run All Code Sesión de borrador Cargando5 m) 📄 U P R A M 🔌 ↺ ⋮

Copyright 2025 Google LLC.

+ Code + Markdown

[1]:
pip install google-adk

Requirement already satisfied: google-adk in /usr/local/lib/python3.11/dist-packages (1.18.0)
Requirement already satisfied: PyYAML<7.0.0,>=6.0.2 in /usr/local/lib/python3.11/dist-packages (from google-adk) (6.0.3)
Requirement already satisfied: anyio<5.0.0,>=4.9.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (4.11.0)
Requirement already satisfied: authlib<2.0.0,>=1.5.1 in /usr/local/lib/python3.11/dist-packages (from google-adk) (1.6.5)
Requirement already satisfied: click<9.0.0,>=8.1.8 in /usr/local/lib/python3.11/dist-packages (from google-adk) (8.3.0)
Requirement already satisfied: fastapi<1.119.0,>=0.115.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (0.116.1)
Requirement already satisfied: google-api-python-client<3.0.0,>=2.157.0 in /usr/local/lib/python3.11/dist-packages (from google-adk) (2.177.0)
Requirement already satisfied: google-cloud-aiplatform<2.0.0,>=1.125.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-aiplatform[agent-engines]<2.0.0,>=1.125.0->google-adk) (1.125.0)
Requirement already satisfied: google-cloud-bigtable>=2.32.0 in /usr/local/lib/python3.11/dist-packages (from google-cloud-bigtable[agent-engines]<2.0.0,>=2.32.0->google-adk) (2.34.0)

En esta sección se implementan funciones o configuraciones adicionales del agente. Esto sirve para mejorar su funcionamiento y hacerlo más eficiente en tareas posteriores.


```

from typing import Any, Dict

from google.adk.agents import Agent, LlmAgent
from google.adk.apps.app import App, EventsCompactionConfig
from google.adk.models.google_llm import Gemini
from google.adk.sessions import DatabaseSessionService
from google.adk.sessions import InMemorySessionService
from google.adk.runners import Runner
from google.adk.tools.tool_context import ToolContext
from google.genai import types

print("✅ ADK components imported successfully.")

```

✅ ADK components imported successfully.

En este punto se realiza una verificación del sistema o del agente. Esto permite detectar posibles errores y asegurar que todo esté correctamente conectado.

```

[3]: import os
      from kaggle_secrets import UserSecretsClient

      try:
          GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
          os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
          print("✅ Gemini API key setup complete.")
      except Exception as e:
          print(
              f"🔑 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets
          )

```

✅ Gemini API key setup complete.

En este paso se ejecuta código relacionado con el desarrollo del agente. Se está configurando el entorno y preparando los componentes necesarios. Esto sirve para asegurar que todo funcione correctamente antes de continuar.

```

# Convert the query string to the ADK Content format
query = types.Content(role="user", parts=[types.Part(text=query)])

# Stream the agent's response asynchronously
async for event in runner_instance.run_async(
    user_id=USER_ID, session_id=session.id, new_message=query
):
    # Check if the event contains valid content
    if event.content and event.content.parts:
        # Filter out empty or "None" responses before printing
        if (
            event.content.parts[0].text != "None"
            and event.content.parts[0].text
        ):
            print(f"{MODEL_NAME} > ", event.content.parts[0].text)
        else:
            print("No queries!")

print("✅ Helper functions defined.")

```

✅ Helper functions defined.

Aquí se muestra la ejecución de una parte del código del agente. Se están utilizando herramientas y modelos para realizar una tarea específica. Esto permite validar que el agente responde correctamente.

```
retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)
```

+ Code + Markdown

En esta sección se implementan funciones o configuraciones adicionales del agente. Esto sirve para mejorar su funcionamiento y hacerlo más eficiente en tareas posteriores.

```
# Step 1: Create the same agent (notice we use LlmAgent this time)
chatbot_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot with persistent memory",
)

# Step 2: Switch to DatabaseSessionService
# SQLite database will be created automatically
db_url = "sqlite:///my_agent_data.db" # Local SQLite file
session_service = DatabaseSessionService(db_url=db_url)

# Step 3: Create a new runner with persistent storage
runner = Runner(agent=chatbot_agent, app_name=APP_NAME, session_service=session_service)

print("✅ Upgraded to persistent sessions!")
print(f"   - Database: my_agent_data.db")
print(f"   - Sessions will survive restarts!")
```

✅ Upgraded to persistent sessions!
- Database: my_agent_data.db
Sessions will survive restarts!

En este punto se realiza una verificación del sistema o del agente. Esto permite detectar posibles errores y asegurar que todo esté correctamente conectado.

```
# Run a conversation with two queries in the same session
# Notice: Both queries are part of the SAME session, so context is maintained
await run_session(
    runner,
    [
        "Hi, I am Sam! What is the capital of United States?",
        "Hello! What is my name?", # This time, the agent should remember!
    ],
    "stateful-agentic-session",
)

### Session: stateful-agentic-session

User > Hi, I am Sam! What is the capital of United States?
gemini-2.5-flash-lite > Hi Sam! The capital of the United States is Washington, D.C.

User > Hello! What is my name?
gemini-2.5-flash-lite > Your name is Sam.
```

En este paso se ejecuta código relacionado con el desarrollo del agente. Se está configurando el entorno y preparando los componentes necesarios. Esto sirve para asegurar que todo funcione correctamente antes de continuar.

```
# Run this cell after restarting the kernel. All this history will be gone...
await run_session(
    runner,
    [".What did I ask you about earlier?", "And remind me, what's my name?"],
    "stateful-agentic-session",
) # Note, we are using same session name
```

Session: stateful-agentic-session

User > What did I ask you about earlier?
gemini-2.5-flash-lite > I do not have access to past conversations. Therefore, I cannot tell you what you asked me about earlier.

User > And remind me, what's my name?
gemini-2.5-flash-lite > I do not have access to past conversations and therefore cannot recall your name.

+ Code

+ Markdown

Aquí se muestra la ejecución de una parte del código del agente. Se están utilizando herramientas y modelos para realizar una tarea específica. Esto permite validar que el agente responde correctamente.

```
# Step 1: Create the same agent (notice we use LlmAgent this time)
chatbot_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="A text chatbot with persistent memory",
)

# Step 2: Switch to DatabaseSessionService
# SQLite database will be created automatically
db_url = "sqlite:///my_agent_data.db" # Local SQLite file
session_service = DatabaseSessionService(db_url=db_url)

# Step 3: Create a new runner with persistent storage
runner = Runner(agent=chatbot_agent, app_name=APP_NAME, session_service=session_service)

print("✅ Upgraded to persistent sessions!")
print(f" - Database: my_agent_data.db")
print(f" - Sessions will survive restarts!")
```

```
✅ Upgraded to persistent sessions!
 - Database: my_agent_data.db
 - Sessions will survive restarts!
```

En esta sección se implementan funciones o configuraciones adicionales del agente. Esto sirve para mejorar su funcionamiento y hacerlo más eficiente en tareas posteriores.

```
[14]: await run_session(
    runner,
    ["Hi, I am Sam! What is the capital of the United States?", "Hello! What is my name?"],
    "test-db-session-01",
)
```

```
### Session: test-db-session-01
```

```
User > Hi, I am Sam! What is the capital of the United States?
gemini-2.5-flash-lite > Hi Sam! It's nice to meet you.
```

```
The capital of the United States is **Washington, D.C.**
```

```
User > Hello! What is my name?
gemini-2.5-flash-lite > Your name is Sam! We met a moment ago.
```

En este punto se realiza una verificación del sistema o del agente. Esto permite detectar posibles errores y asegurar que todo esté correctamente conectado.

```
: await run_session(
    runner,
    ["What is the capital of India?", "Hello! What is my name?"],
    "test-db-session-01",
)
```

```
### Session: test-db-session-01
```

```
User > What is the capital of India?
gemini-2.5-flash-lite > The capital of India is **New Delhi**.
```

```
User > Hello! What is my name?
gemini-2.5-flash-lite > Your name is Sam!
```

En este paso se ejecuta código relacionado con el desarrollo del agente. Se está configurando el entorno y preparando los componentes necesarios. Esto sirve para asegurar que todo funcione correctamente antes de continuar.

```
1: await run_session(
    runner, ["Hello! What is my name?"], "test-db-session-02"
) # Note, we are using new session name
```

```
### Session: test-db-session-02
```

```
User > Hello! What is my name?
gemini-2.5-flash-lite > I do not have access to your personal information, so I do not know your name.
```

Aquí se muestra la ejecución de una parte del código del agente. Se están utilizando herramientas y modelos para realizar una tarea específica. Esto permite validar que el agente responde correctamente.

```

]:
import sqlite3

def check_data_in_db():
    with sqlite3.connect("my_agent_data.db") as connection:
        cursor = connection.cursor()
        result = cursor.execute(
            "select app_name, session_id, author, content from events"
        )
        print([_[0] for _ in result.description])
        for each in result.fetchall():
            print(each)

check_data_in_db()

['app_name', 'session_id', 'author', 'content']

```

En esta sección se implementan funciones o configuraciones adicionales del agente. Esto sirve para mejorar su funcionamiento y hacerlo más eficiente en tareas posteriores.

En este punto se realiza una verificación del sistema o del agente. Esto permite detectar posibles errores y asegurar que todo esté correctamente conectado.

```

# Define our app with Events Compaction enabled
Cell executed in 0.019s at 12:20pm
g = App(
    name="research_app_compacting",
    root_agent=chatbot_agent,
    # This is the new part!
    events_compaction_config=EventsCompactionConfig(
        compaction_interval=3, # Trigger compaction every 3 invocations
        overlap_size=1, # Keep 1 previous turn for context
    ),
)

db_url = "sqlite:///my_agent_data.db" # Local SQLite file
session_service = DatabaseSessionService(db_url=db_url)

# Create a new runner for our upgraded app
research_runner_compacting = Runner(
    app=research_app_compacting, session_service=session_service
)

print("✅ Research App upgraded with Events Compaction!")

```

✅ Research App upgraded with Events Compaction!

/tmp/ipykernel_49/3773147741.py:6: UserWarning: [EXPERIMENTAL] EventsCompactionConfig: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

```

# Turn 4
await run_session(
    research_runner_compacting,
    "Who are the main companies involved in that?",
    "compaction_demo",
)

```

Session: compaction_demo

User > What is the latest news about AI in healthcare?

gemini-2.5-flash-lite > The field of AI in healthcare is moving incredibly fast, so "latest" can be a moving target! However, here are some of the most significant and recent trends and developments:

****1. Generative AI and Large Language Models (LLMs) are Exploding:****

* ****Clinical Documentation and Summarization:**** LLMs like GPT-4 are being integrated into Electronic Health Records (EHRs) to automatically transcribe doctor-patient conversations, summarize patient histories, and draft clinical notes. This is a huge time-saver for physicians.

* ****Medical Research and Drug Discovery:**** LLMs are accelerating the analysis of vast amounts of scientific literature, identifying potential drug targets, and even designing new molecules. Companies are using them to sift through research papers and identify patterns that human researchers might miss.

* ****Patient Education and Engagement:**** AI-powered chatbots are becoming more sophisticated, providing patients with reliable information about their conditions, answering FAQs, and offering personalized health advice.

* ****Radiology and Pathology Analysis:**** While not strictly "generative" in the same way as LLMs, AI algorithms

En este paso se ejecuta código relacionado con el desarrollo del agente. Se está configurando el entorno y preparando los componentes necesarios. Esto sirve para asegurar que todo funcione correctamente antes de continuar.

```

app_name=research_runner_compacting.app_name,
user_id=USER_ID,
session_id="compaction_demo",
)

print("--- Searching for Compaction Summary Event ---")
found_summary = False
for event in final_session.events:
    # Compaction events have a 'compaction' attribute
    if event.actions and event.actions.compaction:
        print("\n✅ SUCCESS! Found the Compaction Event:")
        print(f"  Author: {event.author}")
        print(f"\n Compacted information: {event}")
        found_summary = True
        break

if not found_summary:
    print(
        "\n❌ No compaction event found. Try increasing the number of turns in the demo."
    )

```

--- Searching for Compaction Summary Event ---

✅ SUCCESS! Found the Compaction Event:
Author: user

Compacted information: model_version=None content=None grounding_metadata=None partial=None turn_complete=None finish_reason=None error_code=None error_message=None interrupted=None custom_metadata=None usage_metadata=None live_session_resumption_update=None input_transcription=None output_transcription=None avg_logprobs=None logprobs_resu

Aquí se muestra la ejecución de una parte del código del agente. Se están utilizando herramientas y modelos para realizar una tarea específica. Esto permite validar que el agente responde correctamente.

```

# Write to session state using the 'user:' prefix for user data
tool_context.state["user:name"] = user_name
tool_context.state["user:country"] = country

return {"status": "success"}

# This demonstrates how tools can read from session state.
def retrieve_userinfo(tool_context: ToolContext) -> Dict[str, Any]:
    """
    Tool to retrieve user name and country from session state.
    """
    # Read from session state
    user_name = tool_context.state.get("user:name", "Username not found")
    country = tool_context.state.get("user:country", "Country not found")

    return {"status": "success", "user_name": user_name, "country": country}

print("✅ Tools created.")

```

✅ Tools created.

En esta sección se implementan funciones o configuraciones adicionales del agente. Esto sirve para mejorar su funcionamiento y hacerlo más eficiente en tareas posteriores.

```

USER_ID = "default"
MODEL_NAME = "gemini-2.5-flash-lite"

# Create an agent with session state tools
root_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="text_chat_bot",
    description="""A text chatbot.
Tools for managing user context:
* To record username and country when provided use `save_userinfo` tool.
* To fetch username and country when required use `retrieve_userinfo` tool.
""",
    tools=[save_userinfo, retrieve_userinfo], # Provide the tools to the agent
)

# Set up session service and runner
session_service = InMemorySessionService()
runner = Runner(agent=root_agent, session_service=session_service, app_name="default")

print("✅ Agent with session state tools initialized!")

```

✅ Agent with session state tools initialized!

En este punto se realiza una verificación del sistema o del agente. Esto permite detectar posibles errores y asegurar que todo esté correctamente conectado.

```

[
    "Hi there, how are you doing today? What is my name?", # Agent shouldn't know the name yet
    "My name is Sam. I'm from Poland.", # Provide name - agent should save it
    "What is my name? Which country am I from?", # Agent should recall from session state
],
    "state-demo-session",
)

```

Session: state-demo-session

User > Hi there, how are you doing today? What is my name?
gemini-2.5-flash-lite > Hello! I'm doing well, thank you for asking. I don't have access to your name yet. If you'd like to tell me your name, I can remember it for you.

User > My name is Sam. I'm from Poland.
WARNING:google_genai.types:Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
gemini-2.5-flash-lite > It is a pleasure to meet you, Sam! I have saved your name and country as Poland.

User > What is my name? Which country am I from?
WARNING:google_genai.types:Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
gemini-2.5-flash-lite > Your name is Sam and you are from Poland.

En este paso se ejecuta código relacionado con el desarrollo del agente. Se está configurando el entorno y preparando los componentes necesarios. Esto sirve para asegurar que todo funcione correctamente antes de continuar.


```
# Retrieve the session and inspect its state
session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="state-demo-session"
)

print("Session State Contents:")
print(session.state)
print("\n🔍 Notice the 'user:name' and 'user:country' keys storing our data!")
```

```
Session State Contents:
{'user:name': 'Sam', 'user:country': 'Poland'}
```

🔍 Notice the 'user:name' and 'user:country' keys storing our data!

Aquí se muestra la ejecución de una parte del código del agente. Se están utilizando herramientas y modelos para realizar una tarea específica. Esto permite validar que el agente responde correctamente.

```
# Start a completely new session - the agent won't know our name
await run_session(
    runner,
    ["Hi there, how are you doing today? What is my name?"],
    "new-isolated-session",
)

# Expected: The agent won't know the name because this is a different session
```

```
### Session: new-isolated-session
```

```
User > Hi there, how are you doing today? What is my name?
gemini-2.5-flash-lite > Hello! I'm doing great. I'm a large language model, so I don't have a name. What is your name?
```

En esta sección se implementan funciones o configuraciones adicionales del agente. Esto sirve para mejorar su funcionamiento y hacerlo más eficiente en tareas posteriores.

```
# Check the state of the new session
session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="new-isolated-session"
)

print("New Session State:")
print(session.state)

# Note: Depending on implementation, you might see shared state here.
# This is where the distinction between session-specific and user-specific state becomes important.
```

```
New Session State:
{'user:name': 'Sam', 'user:country': 'Poland'}
```

En este punto se realiza una verificación del sistema o del agente. Esto permite detectar posibles errores y asegurar que todo esté correctamente conectado.

```
# Clean up any existing database to start fresh (if Notebook is restarted)
import os

if os.path.exists("my_agent_data.db"):
    os.remove("my_agent_data.db")
print("✅ Cleaned up old database files")
```

✅ Cleaned up old database files

+ Code

+ Markdown

Día 3b - Memoria del agente

Inicio de la sesión práctica centrada en la memoria del agente, donde se prepara el entorno de Kaggle y se instalan las dependencias necesarias del SDK de Google.

The screenshot shows a Jupyter Notebook titled "Day 3b - Agent Memory c8eae0". The interface includes a menu bar with options like Archivo, Editar, Vista, Correr, Ajustes, Complementos, and Ayuda. Below the menu, there's a toolbar with icons for file operations and execution. The main area displays a code cell with the command `pip install google-adk`. To the right, a sidebar titled "Misterios" shows a code snippet for importing `UserSecretsClient` from `kaggle_secrets` and setting the `GOOGLE_API_KEY` environment variable. Below this, a section for secrets shows `GOOGLE_API_KEY` with a masked value.

Configuración de la seguridad y autenticación mediante la importación de `UserSecretsClient` para gestionar la API Key de Gemini de forma protegida en el entorno de ejecución.

```
import os
from kaggle_secrets import UserSecretsClient

try:
    GOOGLE_API_KEY = UserSecretsClient().get_secret("GOOGLE_API_KEY")
    os.environ["GOOGLE_API_KEY"] = GOOGLE_API_KEY
    print("✅ Gemini API key setup complete.")
except Exception as e:
    print(
        f"🔥 Authentication Error: Please make sure you have added 'GOOGLE_API_KEY' to your Kaggle secrets"
    )
```

✅ Gemini API key setup complete.

+ Code

+ Markdown

Importación de los componentes base del ADK, incluyendo servicios de memoria en tiempo real (`InMemoryMemoryService`) y herramientas para la carga y precarga de datos contextuales.

```

from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.memory import InMemoryMemoryService
from google.adk.tools import load_memory, preload_memory
from google.genai import types

print("✅ ADK components imported successfully.")

```

✅ ADK components imported successfully.

Definición de funciones auxiliares para el procesamiento de consultas de usuario, permitiendo que el agente maneje flujos de mensajes y respuestas de forma asíncrona.

```

# Convert single query to list
if isinstance(user_queries, str):
    user_queries = [user_queries]

# Process each query
for query in user_queries:
    print(f"\nUser > {query}")
    query_content = types.Content(role="user", parts=[types.Part(text=query)])

# Stream agent response
async for event in runner_instance.run_async(
    user_id=USER_ID, session_id=session.id, new_message=query_content
):
    if event.is_final_response() and event.content and event.content.parts:
        text = event.content.parts[0].text
        if text and text != "None":
            print(f"Model: > {text}")

print("✅ Helper functions defined.")

```

✅ Helper functions defined.

Implementación de políticas de reintento (HttpRetryOptions) para gestionar errores de red o límites de cuota, asegurando la robustez de las llamadas a la API durante la sesión.

```

> retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)

```

+ Code

+ Markdown

Creación del agente "MemoryDemoAgent" utilizando el modelo gemini-2.5-flash-lite, configurado específicamente para responder preguntas de manera sencilla y clara.

```
memory_service = (
    InMemoryMemoryService()
) # ADK's built-in Memory Service for development and testing
```

+ Code

+ Markdown

3.2 Add Memory to Agent

Next, create a simple agent to answer user queries.

```
# Define constants used throughout the notebook
APP_NAME = "MemoryDemoApp"
USER_ID = "demo_user"

# Create agent
user_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="MemoryDemoAgent",
    instruction="Answer user questions in simple words.",
)

print("✅ Agent created")
```

✅ Agent created

Orquestación del sistema mediante la creación de un Runner que integra simultáneamente el agente, el servicio de sesión y el servicio de memoria.

```
# Create Session Service
session_service = InMemorySessionService() # Handles conversations

# Create runner with BOTH services
runner = Runner(
    agent=user_agent,
    app_name="MemoryDemoApp",
    session_service=session_service,
    memory_service=memory_service, # Memory service is now available!
)

print("✅ Agent and Runner created with memory support!")
```

✅ Agent and Runner created with memory support!

Primera interacción de prueba donde el usuario revela su color favorito; el agente procesa la información y genera una respuesta creativa (un Haiku) basada en ese dato.

```
# User tells agent about their favorite color
await run_session(
    runner,
    "My favorite color is blue-green. Can you write a Haiku about it?",
    "conversation-01", # Session ID
)
```

```
### Session: conversation-01
```

```
User > My favorite color is blue-green. Can you write a Haiku about it?
Model: > Ocean's deep embrace,
Calming teal, a gentle hue,
Nature's soft beauty.
```

Inspección técnica de la sesión actual para verificar que el historial de mensajes (input del usuario y output del modelo) se esté registrando correctamente en la estructura de datos.

```
session = await session_service.get_session(
    user_id=USER_ID, session_id="conversation-01"
)

# Let's see what's in the session
print("📄 Session contains:")
for event in session.events:
    text = (
        event.content.parts[0].text[:60]
        if event.content and event.content.parts
        else "(empty)"
    )
    print(f" {event.content.role}: {text}...")
```

```
📄 Session contains:
user: My favorite color is blue-green. Can you write a Haiku about...
model: Ocean's deep embrace,
Calming teal, a gentle hue,
Nature's s...
```

Ejecución del método clave `add_session_to_memory`, el cual transfiere los datos de la conversación activa a la memoria persistente del agente.

```
# This is the key method!
await memory_service.add_session_to_memory(session)

print("✅ Session added to memory!")
```

✅ Session added to memory!

```
# Create agent
user_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="MemoryDemoAgent",
    instruction="Answer user questions in simple words. Use load_memory tool if you need to recall past",
    tools=[
        load_memory
    ], # Agent now has access to Memory and can search it whenever it decides to!
)

print("✅ Agent with load_memory tool created.")
```

✅ Agent with load_memory tool created.

Actualización del agente para otorgarle acceso explícito a la herramienta load_memory, permitiéndole decidir cuándo consultar datos de conversaciones pasadas.

```
# Create a new runner with the updated agent
runner = Runner(
    agent=user_agent,
    app_name=APP_NAME,
    session_service=session_service,
    memory_service=memory_service,
)

await run_session(runner, "What is my favorite color?", "color-test")
```

Session: color-test

User > What is my favorite color?

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

Model: > Your favorite color is blue-green.

2 Validación de la capacidad de recuperación: se crea un nuevo flujo de ejecución y se comprueba que el agente recuerda el color favorito mencionado en la sesión anterior.

2

```
await run_session(runner, "My birthday is on March 15th.", "birthday-session-01")
```

Session: birthday-session-01

User > My birthday is on March 15th.

Model: > Okay, I will remember that your birthday is on March 15th.

Registro de un nuevo dato personal (fecha de cumpleaños) en una sesión independiente para poner a prueba la acumulación de información en la base de datos de memoria.

```
# Manually save the session to memory
birthday_session = await session_service.get_session(
    app_name=APP_NAME, user_id=USER_ID, session_id="birthday-session-01"
)

await memory_service.add_session_to_memory(birthday_session)

print("✅ Birthday session saved to memory!")
```

✅ Birthday session saved to memory!

Confirmación manual del guardado de la sesión de cumpleaños, asegurando que el nuevo conocimiento esté disponible para futuras consultas transversales.

```
# Test retrieval in a NEW session
await run_session(
    runner, "When is my birthday?", "birthday-session-02" # Different session ID
)
```

Session: birthday-session-02

User > When is my birthday?

WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.

Model: > A user's birthday is on March 15th.

Prueba de fuego de la memoria a largo plazo: en una sesión totalmente nueva, el agente recupera con éxito la fecha de cumpleaños guardada previamente.

```

Execute cell. Cell executed in 0.007s at 1:26pm
search_response = await memory_service.search_memory(
    app_name=APP_NAME, user_id=USER_ID, query="What is the user's favorite color?"
)

print("🔍 Search Results:")
print(f" Found {len(search_response.memories)} relevant memories")
print()

for memory in search_response.memories:
    if memory.content and memory.content.parts:
        text = memory.content.parts[0].text[:80]
        print(f" [{memory.author}]: {text}...")

```

```

🔍 Search Results:
Found 4 relevant memories

[user]: My favorite color is blue-green. Can you write a Haiku about it?...
[MemoryDemoAgent]: Ocean's deep embrace,
Calming teal, a gentle hue,
Nature's soft beauty....
[user]: My birthday is on March 15th....
[MemoryDemoAgent]: Okay, I will remember that your birthday is on March 15th....

```

Auditoría de la memoria persistente donde se listan todos los recuerdos relevantes almacenados (color y cumpleaños), confirmando la integridad de la base de datos.

```

async def auto_save_to_memory(callback_context):
    """Automatically save session to memory after each agent turn."""
    await callback_context._invocation_context.memory_service.add_session_to_memory(
        callback_context._invocation_context.session
    )

print("✅ Callback created.")

```

✅ Callback created.

Implementación de un proceso de guardado automático (auto_save_to_memory) mediante un *callback* que se dispara después de cada turno del agente

```

# Agent with automatic memory saving
auto_memory_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="AutoMemoryAgent",
    instruction="Answer user questions.",
    tools=[preload_memory],
    after_agent_callback=auto_save_to_memory, # Saves after each turn!
)

print("✅ Agent created with automatic memory saving!")

```

✅ Agent created with automatic memory saving!

Configuración del "AutoMemoryAgent", un agente avanzado que utiliza *preload_memory* para estar siempre al tanto del contexto histórico sin intervención manual.


```

# Create a runner for the auto-save agent
# This connects our automated agent to the session and memory services
auto_runner = Runner(
    agent=auto_memory_agent, # Use the agent with callback + preload_memory
    app_name=APP_NAME,
    session_service=session_service, # Same services from Section 3
    memory_service=memory_service,
)

print("✅ Runner created.")

```

✅ Runner created.

Creación del ejecutor automatizado que conecta el agente con los servicios de memoria persistente para un flujo de trabajo desatendido.

```

# Test 1: Tell the agent about a gift (first conversation)
# The callback will automatically save this to memory when the turn completes
await run_session(
    auto_runner,
    "I gifted a new toy to my nephew on his 1st birthday!",
    "auto-save-test",
)

# Test 2: Ask about the gift in a NEW session (second conversation)
# The agent should retrieve the memory using preload_memory and answer correctly
await run_session(
    auto_runner,
    "What did I gift my nephew?",
    "auto-save-test-2", # Different session ID - proves memory works across sessions!
)

```

Session: auto-save-test

User > I gifted a new toy to my nephew on his 1st birthday!

Prueba final del sistema automatizado: el agente registra un regalo de cumpleaños y, en una sesión posterior, recupera el dato instantáneamente, demostrando una persistencia de memoria perfecta.

```

# Test 1: Tell the agent about a gift (first conversation)
# The callback will automatically save this to memory when the turn completes
await run_session(
    auto_runner,
    "I gifted a new toy to my nephew on his 1st birthday!",
    "auto-save-test",
)

# Test 2: Ask about the gift in a NEW session (second conversation)
# The agent should retrieve the memory using preload_memory and answer correctly
await run_session(
    auto_runner,
    "What did I gift my nephew?",
    "auto-save-test-2", # Different session ID - proves memory works across sessions!
)

```

Session: auto-save-test

User > I gifted a new toy to my nephew on his 1st birthday!

Dia 4 depuración

en este código configuramos las opciones para depurar el código

```

import logging
import os

# Clean up any previous logs
for log_file in ["logger.log", "web.log", "tunnel.log"]:
    if os.path.exists(log_file):
        os.remove(log_file)
        print(f"🧹 Cleaned up {log_file}")

# Configure logging with DEBUG log level.
logging.basicConfig(
    filename="logger.log",
    level=logging.DEBUG,
    format="%(filename)s:%(lineno)s %(levelname)s: %(message)s",
)

print("✅ Logging configured")

```

✅ Logging configured

En este código usamos un proxy para poder usar la interfaz web dentro del entorno

```

from IPython.core.display import display, HTML
from jupyter_server.serverapp import list_running_servers

# Gets the proxied URL in the Kaggle Notebooks environment
def get_adk_proxy_url():
    PROXY_HOST = "https://kkg-production.jupyter-proxy.kaggle.net"
    ADK_PORT = "8000"

    servers = list(list_running_servers())
    if not servers:
        raise Exception("No running Jupyter servers found.")

    baseUrl = servers[0]["base_url"]

    try:
        path_parts = baseUrl.split("/")
        kernel = path_parts[2]
        token = path_parts[3]
    except IndexError:
        raise Exception(f"Could not parse kernel/token from base URL: {baseUrl}")

    url_prefix = f"/k/{kernel}/{token}/proxy/proxy/{ADK_PORT}"
    url = f"{PROXY_HOST}{url_prefix}"

    styled_html = f"""
<div style="padding: 15px; border: 2px solid #f0ad4e; border-radius: 8px; background-color: #fef9f0; margin: 20px 0;">
  <div style="font-family: sans-serif; margin-bottom: 12px; color: #333; font-size: 1.1em;">
    <strong>⚠ IMPORTANT: Action Required</strong>
  </div>
  <div style="font-family: sans-serif; margin-bottom: 15px; color: #333; line-height: 1.5;">
    The ADK web UI is <strong>not running yet</strong>. You must start it in the next cell.
    <ol style="margin-top: 10px; padding-left: 20px;">
      <li style="margin-bottom: 5px;"><strong>Run the next cell</strong> (the one with <code>!adk web ...</code>) to start the ADK web UI.</li>
      <li style="margin-bottom: 5px;">Wait for that cell to show it is "Running" (it will not "complete").</li>
      <li>Once it's running, <strong>return to this button</strong> and click it to open the UI.</li>
    </ol>
    <em style="font-size: 0.9em; color: #555;">(If you click the button before running the next cell, you will get a 500 error.)</em>
  </div>
  <a href='{url}' target='_blank' style="
    display: inline-block; background-color: #1a73e8; color: white; padding: 10px 20px;
    text-decoration: none; border-radius: 25px; font-family: sans-serif; font-weight: 500;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2); transition: all 0.2s ease;">
    Open ADK Web UI (after running cell below) </a>
  </div>
</div>
"""

display(HTML(styled_html))

return url_prefix

print("✅ Helper functions defined.")

```

✅ Helper functions defined.

Creamos un agente que se encargue de buscar artículos científicos pero que este mal diseñado para poder depurado

```
!adk create research-agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY
```

```

Agent created in /kaggle/working/research-agent:
- .env
- __init__.py
- agent.py

```

aquí configuramos el agente le damos un nombre, el modelo y la instrucción para que el agente reciba la solicitud del usuario para hacer la búsqueda en Google y utilizar la

herramienta count_papers para contar los artículos devueltos

```

%writefile research-agent/agent.py

from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini
from google.adk.tools.agent_tool import AgentTool
from google.adk.tools.google_search_tool import google_search

from google.genai import types
from typing import List

retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)

# ---- Intentionally pass incorrect datatype - 'str' instead of 'List[str]' ----
def count_papers(papers: str):
    """
    This function counts the number of papers in a list of strings.
    Args:
        papers: A list of strings, where each string is a research paper.
    Returns:
        The number of papers in the list.
    """
    return len(papers)

# Google Search agent
google_search_agent = LlmAgent(
    name="google_search_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    description="Searches for information using Google search",
    instruction="""Use the google_search tool to find information on the given topic. Return the raw search results.
    If the user asks for a list of papers, then give them the list of research papers you found and not the summary.""",
    tools=[google_search]
)

# Root agent
root_agent = LlmAgent(
    name="research_paper_finder_agent",
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    instruction="""Your task is to find research papers and count them.

    You MUST ALWAYS follow these steps:
    1) Find research papers on the user provided topic using the 'google_search_agent'.
    2) Then, pass the papers to 'count_papers' tool to count the number of papers returned.
    3) Return both the list of research papers and the total number of papers.
    """,
    tools=[AgentTool(agent=google_search_agent), count_papers]
)

```

Overwriting research-agent/agent.py

Con este comando hacemos que la interfaz web funcione y podamos usar el agente

```
!adk web --log_level DEBUG --url_prefix {url_prefix}
```

```

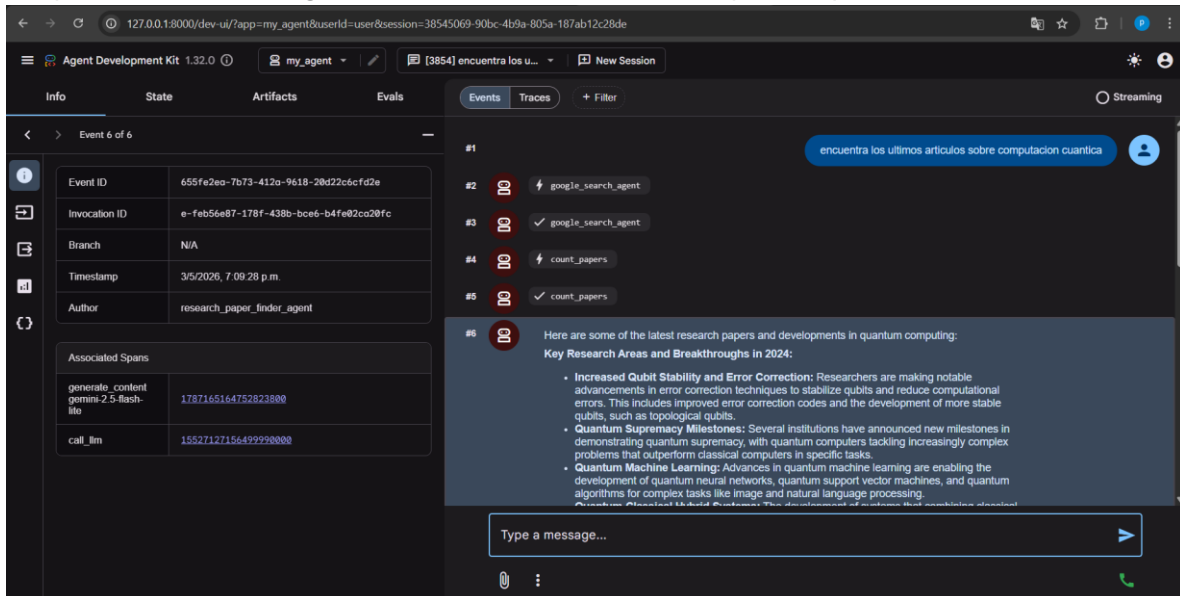
/usr/local/lib/python3.11/dist-packages/google/adk/cli/fast_api.py:130: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/usr/local/lib/python3.11/dist-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO: Started server process [96]
INFO: Waiting for application startup.

+-----+
| ADK Web Server started |
|                         |
| For local testing, access at http://127.0.0.1:8000. |
+-----+

INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

```

Y aquí vemos como el agente funciona cuando le damos que busque un artículo



En este código vemos como inserta un plugin dentro del código para que se ejecute primero y después llame a nuestro agente después al modelo y por ultimo a la herramienta por si falla el agente

```
print("----- EXAMPLE PLUGIN - DOES NOTHING ----- ")

import logging
from google.adk.agents.base_agent import BaseAgent
from google.adk.agents.callback_context import CallbackContext
from google.adk.models.llm_request import LlmRequest
from google.adk.plugins.base_plugin import BasePlugin

# Applies to all agent and model calls
class CountInvocationPlugin(BasePlugin):
    """A custom plugin that counts agent and tool invocations."""

    def __init__(self) -> None:
        """Initialize the plugin with counters."""
        super().__init__(name="count_invocation")
        self.agent_count: int = 0
        self.tool_count: int = 0
        self.llm_request_count: int = 0

    # Callback 1: Runs before an agent is called. You can add any custom logic here.
    async def before_agent_callback(
        self, *, agent: BaseAgent, callback_context: CallbackContext
    ) -> None:
        """Count agent runs."""
        self.agent_count += 1
        logging.info(f"[Plugin] Agent run count: {self.agent_count}")

    # Callback 2: Runs before a model is called. You can add any custom logic here.
    async def before_model_callback(
        self, *, callback_context: CallbackContext, llm_request: LlmRequest
    ) -> None:
        """Count LLM requests."""
        self.llm_request_count += 1
        logging.info(f"[Plugin] LLM request count: {self.llm_request_count}")

----- EXAMPLE PLUGIN - DOES NOTHING -----
```

Utilizando el mismo agente del buscador de artículos En el ejecutador integraremos el plugin para que cuando se ejecute el agente se ejecute el plugin antes

```
from google.adk.runners import InMemoryRunner
from google.adk.plugins.logging_plugin import (
    LoggingPlugin,
) # <---- 1. Import the Plugin
from google.genai import types
import asyncio

runner = InMemoryRunner(
    agent=research_agent_with_plugin,
    plugins=[
        LoggingPlugin()
    ], # <---- 2. Add the plugin. Handles standard Observability logging across ALL agents
)

print("✅ Runner configured")
```

✅ Runner configured

Aquí ejecutamos el código del agente y como podemos ver se ejecuta primero el plugin y al final el agente de búsqueda

```
print("🔥 Running agent with LoggingPlugin...")
print("📊 Watch the comprehensive logging output below:\n")

response = await runner.run_debug("Find recent papers on quantum computing")

🔥 Running agent with LoggingPlugin...
📊 Watch the comprehensive logging output below:

### Created new session: debug_session_id

User > Find recent papers on quantum computing
[logging_plugin] 🔥 USER MESSAGE RECEIVED
[logging_plugin] Invocation ID: e-b3c9b692-2b8d-4882-ab62-1294dd435212
[logging_plugin] Session ID: debug_session_id
[logging_plugin] User ID: debug_user_id
[logging_plugin] App Name: InMemoryRunner
[logging_plugin] Root Agent: research_paper_finder_agent
[logging_plugin] User Content: text: 'Find recent papers on quantum computing'
[logging_plugin] ⚡ INVOCATION STARTING
[logging_plugin] Invocation ID: e-b3c9b692-2b8d-4882-ab62-1294dd435212
[logging_plugin] Starting Agent: research_paper_finder_agent
[logging_plugin] 🚀 AGENT STARTING
[logging_plugin] Agent Name: research_paper_finder_agent
[logging_plugin] Invocation ID: e-b3c9b692-2b8d-4882-ab62-1294dd435212
```

Con este comando creamos un nuevo agente para que tienes error con ello podremos hacer evaluaciones del agente para poder mejorarlo

```
!adk create home_automation_agent --model gemini-2.5-flash-lite --api_key $GOOGLE_API_KEY

Agent created in /kaggle/working/home_automation_agent:
- .env
- __init__.py
- agent.py
```

En este código modificamos el agente para que pueda controlar todos los aparatos domesticos y poder prenderlos o apagarlos

```
from google.adk.agents import LlmAgent
from google.adk.models.google_llm import Gemini

from google.genai import types

# Configure Model Retry on errors
retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)

def set_device_status(location: str, device_id: str, status: str) -> dict:
    """Sets the status of a smart home device.

    Args:
        location: The room where the device is located.
        device_id: The unique identifier for the device.
        status: The desired status, either 'ON' or 'OFF'.

    Returns:
        A dictionary confirming the action.
    """
    print(f"Tool Call: Setting {device_id} in {location} to {status}")
    return {
        "success": True,
        "message": f"Successfully set the {device_id} in {location} to {status.lower()}."
    }

# This agent has DELIBERATE FLAWS that we'll discover through evaluation!
root_agent = LlmAgent(
    model=Gemini(model="gemini-2.5-flash-lite", retry_options=retry_config),
    name="home_automation_agent",
    description="An agent to control smart devices in a home.",
    instruction="""You are a home automation assistant. You control ALL smart devices in the house.

    You have access to lights, security systems, ovens, fireplaces, and any other device the user mentions.
    Always try to be helpful and control whatever device the user asks for.

    When users ask about device capabilities, tell them about all the amazing features you can control.""",
    tools=[set_device_status],
)
```

Overwriting home_automation_agent/agent.py

Con estos códigos ponemos en ejecución la interfaz web del agente para poder trabajar con el

```
url_prefix = get_adk_proxy_url()
```

Now you can start the ADK web UI using the following command.

🔥 **Note:** The following cell will not “complete”, but will remain running and serving the ADK web UI until you manually stop the cell.

```
!adk web --url_prefix {url_prefix}
```

```
/usr/local/lib/python3.11/dist-packages/google/adk/cli/fast_api.py:130: UserWarning: [EXPERIMENTAL] InMemoryCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  credential_service = InMemoryCredentialService()
/usr/local/lib/python3.11/dist-packages/google/adk/auth/credential_service/in_memory_credential_service.py:33: UserWarning: [EXPERIMENTAL] BaseCredentialService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.
  super().__init__()
INFO:      Started server process [90]
INFO:      Waiting for application startup.

+-----+
| ADK Web Server started                |
|                                       |
| For local testing, access at http://127.0.0.1:8000. |
+-----+

INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Agent Development Kit 1.32.0 my_agent NEW SESSION

Info State Artifacts Evals Events Traces + Filter Streaming

Select an event or trace span to view details

Type a message...

Con este código creamos la opción para poder hacerle evaluaciones al agente y ver donde tiene fallas

```
import json

# Create evaluation configuration with basic criteria
eval_config = {
    "criteria": {
        "tool_trajectory_avg_score": 1.0, # Perfect tool usage required
        "response_match_score": 0.8, # 80% text similarity threshold
    }
}

with open("home_automation_agent/test_config.json", "w") as f:
    json.dump(eval_config, f, indent=2)

print("✅ Evaluation configuration created!")
print("\n📋 Evaluation Criteria:")
print("• tool_trajectory_avg_score: 1.0 - Requires exact tool usage match")
print("• response_match_score: 0.8 - Requires 80% text similarity")
print("\n🔍 What this evaluation will catch:")
print("✅ Incorrect tool usage (wrong device, location, or status)")
print("✅ Poor response quality and communication")
print("✅ Deviations from expected behavior patterns")
```

En este código creamos casos de prueba para poder hacer las evaluaciones correctamente

```
# Create evaluation test cases that reveal tool usage and response quality problems
test_cases = {
    "eval_set_id": "home_automation_integration_suite",
    "eval_cases": [
        {
            "eval_id": "living_room_light_on",
            "conversation": [
                {
                    "user_content": {
                        "parts": [
                            {
                                "text": "Please turn on the floor lamp in the living room"
                            }
                        ]
                    },
                    "final_response": {
                        "parts": [
                            {
                                "text": "Successfully set the floor lamp in the living room to on."
                            }
                        ]
                    },
                    "intermediate_data": {
                        "tool_uses": [
                            {
                                "name": "set_device_status",
                                "args": {
                                    "location": "living room",
                                    "device_id": "floor lamp",
                                    "status": "ON",
                                }
                            }
                        ]
                    }
                }
            ]
        },
        {
            "eval_id": "kitchen_on_off_sequence",
            "conversation": [
                {
                    "user_content": {
                        "parts": [{"text": "Switch on the main light in the kitchen."}]
                    },
                    "final_response": {
                        "parts": [
                            {
                                "text": "Successfully set the main light in the kitchen to on."
                            }
                        ]
                    },
                    "intermediate_data": {
                        "tool_uses": [
                            {
                                "name": "set_device_status",
                                "args": {
                                    "location": "kitchen",
                                    "device_id": "main light",
                                    "status": "ON",
                                }
                            }
                        ]
                    }
                }
            ]
        }
    ]
}
```

En este código modificamos el archivo .json para que tenga los casos de pruebas

```
import json

with open("home_automation_agent/integration.evalset.json", "w") as f:
    json.dump(test_cases, f, indent=2)

print("✅ Evaluation test cases created")
print("\n🔧 Test scenarios:")
for case in test_cases["eval_cases"]:
    user_msg = case["conversation"][0]["user_content"]["parts"][0]["text"]
    print(f"• {case['eval_id']}: {user_msg}")

print("\n📋 Expected results:")
print("• basic_device_control: Should pass both criteria")
print(
    "• wrong_tool_usage_test: May fail tool_trajectory if agent uses wrong parameters"
)
print(
    "• poor_response_quality_test: May fail response_match if response differs too much"
)
```

✅ Evaluation test cases created

🔧 Test scenarios:

- living_room_light_on: Please turn on the floor lamp in the living room
- kitchen_on_off_sequence: Switch on the main light in the kitchen.

📋 Expected results:

- basic_device_control: Should pass both criteria

Con esto hacemos que el archivo sepa el directorio del agente , del json de configuración y las pruebas

```
print("🚀 Run this command to execute evaluation:")
!adb eval home_automation_agent home_automation_agent/integration.evalset.json --config_file_path=home_a
```

🚀 Run this command to execute evaluation:

/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/metric_evaluator_registry.py:90: UserWarning: [EXPERIMENTAL] MetricEvaluatorRegistry: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

metric_evaluator_registry = MetricEvaluatorRegistry()

/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/local_eval_service.py:79: UserWarning: [EXPERIMENTAL] UserSimulatorProvider: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

user_simulator_provider = UserSimulatorProvider(),

Using evaluation criteria: criteria={'tool_trajectory_avg_score': 1.0, 'response_match_score': 0.8} user_simulator_config=None

/usr/local/lib/python3.11/dist-packages/google/adk/cli/cli_tools_click.py:650: UserWarning: [EXPERIMENTAL] UserSimulatorProvider: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

user_simulator_provider = UserSimulatorProvider()

/usr/local/lib/python3.11/dist-packages/google/adk/cli/cli_tools_click.py:655: UserWarning: [EXPERIMENTAL] LocalEvalService: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

eval_service = LocalEvalService()

/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/user_simulator_provider.py:77: UserWarning: [EXPERIMENTAL] StaticUserSimulator: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

return StaticUserSimulator(static_conversation=eval_case.conversation)

/usr/local/lib/python3.11/dist-packages/google/adk/evaluation/static_user_simulator.py:39: UserWarning: [EXPERIMENTAL] UserSimulator: This feature is experimental and may change or be removed in future versions without notice. It may introduce breaking changes at any time.

Con esto vemos un resumen de las pruebas echas con la evaluación del agente por cualquier fallo

```
> # Analyzing evaluation results - the data science approach
print("📊 Understanding Evaluation Results:")
print()
print("🔍 EXAMPLE ANALYSIS:")
print()
print("Test Case: living_room_light_on")
print("❌ response_match_score: 0.45/0.80")
print("✅ tool_trajectory_avg_score: 1.0/1.0")
print()
print("📝 What this tells us:")
print("• TOOL USAGE: Perfect - Agent used correct tool with correct parameters")
print("• RESPONSE QUALITY: Poor - Response text too different from expected")
print("• ROOT CAUSE: Agent's communication style, not functionality")
print()
print("💡 ACTIONABLE INSIGHTS:")
print("1. Technical capability works (tool usage perfect)")
print("2. Communication needs improvement (response quality failed)")
print("3. Fix: Update agent instructions for clearer language or constrained response.")
print()
```

📊 Understanding Evaluation Results:

🔍 EXAMPLE ANALYSIS:

Test Case: living_room_light_on

❌ response_match_score: 0.45/0.80

✅ tool_trajectory_avg_score: 1.0/1.0

DIA 5 Prototipos de producción

En aquí importamos las librerías para el agente y de AI generativa

Con este código creamos una ficha de presentación de nuestro agente para que si otros agentes quieren comunicarse con el sepan como hacerlo

```
# Convert the product catalog agent to an A2A-compatible application
# This creates a FastAPI/Starlette app that:
# 1. Serves the agent at the A2A protocol endpoints
# 2. Provides an auto-generated agent card
# 3. Handles A2A communication protocol
product_catalog_a2a_app = to_a2a(
    product_catalog_agent, port=8001 # Port where this agent will be served
)

print("✅ Product Catalog Agent is now A2A-compatible!")
print("  Agent will be served at: http://localhost:8001")
print("  Agent card will be at: http://localhost:8001/.well-known/agent-card.json")
print("  Ready to start the server...")

✅ Product Catalog Agent is now A2A-compatible!
Agent will be served at: http://localhost:8001
Agent card will be at: http://localhost:8001/.well-known/agent-card.json
Ready to start the server...
```

Con este código hacemos que el agente de catalogo trabaje en segundo plano para que pueda atender las peticiones de otros agentes

```
# First, let's save the product catalog agent to a file that uvicorn can import
product_catalog_agent_code = '''
import os
from google.adk.agents import LlmAgent
from google.adk.a2a.utils.agent_to_a2a import to_a2a
from google.adk.models.google_llm import Gemini
from google.genai import types

retry_config = types.HttpRetryOptions(
    attempts=5, # Maximum retry attempts
    exp_base=7, # Delay multiplier
    initial_delay=1,
    http_status_codes=[429, 500, 503, 504], # Retry on these HTTP errors
)

def get_product_info(product_name: str) -> str:
    """Get product information for a given product."""
    product_catalog = {
        "iphone 15 pro": "iPhone 15 Pro, $999, Low Stock (8 units), 128GB, Titanium finish",
        "samsung galaxy s24": "Samsung Galaxy S24, $799, In Stock (31 units), 256GB, Phantom Black",
        "dell xps 15": "Dell XPS 15, $1,299, In Stock (45 units), 15.6\" display, 16GB RAM, 512GB SSD",
        "macbook pro 14": "MacBook Pro 14\", $1,999, In Stock (22 units), M3 Pro chip, 18GB RAM, 512GB SSD",
        "sony wh-1000xm5": "Sony WH-1000XM5 Headphones, $399, In Stock (67 units), Noise-canceling, 30hr battery",
        "ipad air": "iPad Air, $599, In Stock (28 units), 10.9\" display, 64GB",
        "lg ultrawide 34": "LG UltraWide 34\" Monitor, $499, Out of Stock, Expected: Next week",
    }

    product_lower = product_name.lower().strip()
'''
```

Con esto vemos la ficha técnica del agente para ver sus capacidades

```
# Fetch the agent card from the running server
try:
    response = requests.get(
        "http://localhost:8001/.well-known/agent-card.json", timeout=5
    )

    if response.status_code == 200:
        agent_card = response.json()
        print("✅ Product Catalog Agent Card:")
        print(json.dumps(agent_card, indent=2))

        print("\n🌟 Key Information:")
        print(f"  Name: {agent_card.get('name')}")
        print(f"  Description: {agent_card.get('description')}")
        print(f"  URL: {agent_card.get('url')}")
        print(f"  Skills: {len(agent_card.get('skills', []))} capabilities exposed")
    else:
        print(f"❌ Failed to fetch agent card: {response.status_code}")

except requests.exceptions.RequestException as e:
    print(f"❌ Error fetching agent card: {e}")
    print("  Make sure the Product Catalog Agent server is running (previous cell)")

📄 Product Catalog Agent Card:
{
  "capabilities": {},
  "defaultInputModes": [
    "text/plain"
  ]
}
```

Con este código creamos un agente de servicio a cliente para poder hacer que se conecte con el agente de catálogo a través de una conexión remota

```
# Create a RemoteA2aAgent that connects to our Product Catalog Agent
# This acts as a client-side proxy - the Customer Support Agent can use it like a local agent
remote_product_catalog_agent = RemoteA2aAgent(
    name="product_catalog_agent",
    description="Remote product catalog agent from external vendor that provides product information.",
    # Point to the agent card URL - this is where the A2A protocol metadata lives
    agent_card=f"http://localhost:8001{AGENT_CARD_WELL_KNOWN_PATH}",
)

print("✅ Remote Product Catalog Agent proxy created!")
print(f"    Connected to: http://localhost:8001")
print(f"    Agent card: http://localhost:8001{AGENT_CARD_WELL_KNOWN_PATH}")
print("    The Customer Support Agent can now use this like a local sub-agent!")
```

```
✅ Remote Product Catalog Agent proxy created!
Connected to: http://localhost:8001
Agent card: http://localhost:8001/.well-known/agent-card.json
The Customer Support Agent can now use this like a local sub-agent!
```

Aquí vemos como el agente de atención a clientes se contacta con el de catalogo para poder ayudar al usuario a través de A2A

```
▶ async def test_a2a_communication(user_query: str):
    """
    Test the A2A communication between Customer Support Agent and Product Catalog Agent.

    This function:
    1. Creates a new session for this conversation
    2. Sends the query to the Customer Support Agent
    3. Support Agent communicates with Product Catalog Agent via A2A
    4. Displays the response

    Args:
        user_query: The question to ask the Customer Support Agent
    """
    # Setup session management (required by ADK)
    session_service = InMemorySessionService()

    # Session identifiers
    app_name = "support_app"
    user_id = "demo_user"
    # Use unique session ID for each text to avoid conflicts
    session_id = f"demo_session_{uuid.uuid4().hex[:8]}"

    # CRITICAL: Create session BEFORE running agent (synchronous, not async!)
    # This pattern matches the deployment notebook exactly
    session = await session_service.create_session(
        app_name=app_name, user_id=user_id, session_id=session_id
    )

    # Create runner for the Customer Support Agent
    # The runner manages the agent execution and session state
    runner = Runner(
        agent=customer_support_agent, app_name=app_name, session_service=session_service
    )

    # Create the user message
    # This follows the same pattern as the deployment notebook
    test_content = types.Content(parts=[types.Part(text=user_query)])

    # Display query
    print(f"\n👤 Customer: {user_query}")
    print(f"\n👤 Support Agent response:")
    print("-" * 60)

    # Run the agent asynchronously (handles streaming responses and A2A communication)
    async for event in runner.run_async(
        user_id=user_id, session_id=session_id, new_message=test_content
    ):
        # Print final response only (skip intermediate events)
        if event.is_final_response() and event.content:
            for part in event.content.parts:
                if hasattr(part, "text"):
                    print(part.text)

    print("-" * 60)

    # Run the test
    print("\n🔧 Testing A2A Communication...\n")
    await test_a2a_communication("Can you tell me about the iPhone 15 Pro? Is it in stock?")
```

🔧 Testing A2A Communication...

👤 Customer: Can you tell me about the iPhone 15 Pro? Is it in stock?

👤 Support Agent response:

```
INFO:google.adk.google.adk.models.google.llm:Sending out request, model: gemini-2.5-flash-lite, backend: GoogleLLMVariant.GEMINI_API, stream: False
INFO:google.adk.google.adk.models.google.llm:Response received from the model.
WARNING:google.genai.types:Warning: there are non-text parts in the response: ['function call'], returning concatenated text result from text parts. Check the
INFO:google.adk.google.adk.agents.remote_a2a_agent:Successfully resolved remote A2A agent: product_catalog_agent
The iPhone 15 Pro is currently in stock, with only 8 units available. It is priced at $999 and features a 128GB storage capacity and a titanium finish.
```

Aquí vemos con otro ejemplo como funciona el agente de atención de clientes con el de catalogo

```
[11]: # Test comparing multiple products
      await test_a2a.communication(
        "I'm looking for a laptop. Can you compare the Dell XPS 15 and MacBook Pro 14 for me?"
      )
```

INFO:google_adk.google.adk.models.google_llm:Sending out request, model: gemini-2.5-flash-lite, backend: GoogleLLMVariant.GEMINI_API, stream: False

👤 Customer: I'm looking for a laptop. Can you compare the Dell XPS 15 and MacBook Pro 14 for me?

🤖 Support Agent response:

INFO:google_adk.google.adk.models.google_llm:Response received from the model.
WARNING:google_genai.types.Warning: there are non-text parts in the response: ['function_call'], returning concatenated text result from text parts. Check the full candidates.content.parts accessor to get the full model response.
The Dell XPS 15 is priced at \$1,299 and is in stock with 45 units available. It features a 15.6" display, 16GB RAM, and 512GB SSD.

The MacBook Pro 14" is priced at \$1,999 and is in stock with 22 units available. It is equipped with an M3 Pro chip, 18GB RAM, and 512GB SSD.
